

西南交通大学



《云计算技术》

课程报告

报告题目： 基于华为云 CCE 的容器化 Web 应用与
Spark 并行数据分析

学 号： 2023112456 / 2023112447

姓 名： 刘俊豪（2023112456），闫硕（2023112447）

班 级： 计算机 2 班

学院名称： 计算机与人工智能学院

授课教师： 戴朋林、丛荣

目录

实验概述	3
小组分工与环境信息	3
小组分工	3
实验环境	3
评分项覆盖自查	4
第一部分 云计算平台搭建	4
任务 1 应用容器化	4
任务 2 CCE 集群搭建	7
任务 3 应用部署	7
任务 4 Redis 持久化存储	8
任务 5 ConfigMap Volume 挂载	10
任务 6 HPA 弹性伸缩	11
第二部分 Spark 大数据分析	12
A-0 Spark Operator 环境部署	12
A-1 数据清洗	15
A-2 Spark SQL 统计分析	16
A-3 性能对比与 Amdahl 分析	19
附加题记录	20
附加题 1 监控系统	20
附加题 2 CI/CD 流水线	21
附加题 3 K3s + MQTT 边缘计算模拟	23
问题与解决汇总	25
总结与收获	25
附录 代码与仓库说明	26

实验概述

本课程设计围绕“本地容器化验证、华为云 CCE 集群部署、Kubernetes 资源编排、Redis 持久化、ConfigMap 配置管理、HPA 弹性伸缩、Spark 大数据分析与附加实验扩展”展开。小组共两名成员，其中刘俊豪的学号为 2023112456，闫硕的学号为 2023112447。本报告遵循“只记录已验证事实”的原则：正文中的命令、现象、截图和分析均来源于仓库内已有代码、终端输出和小组成员提供的实验截图，不对未执行内容做虚构补充。

本组最终选择第二部分的方向 A，即基于 Spark Operator 在 Kubernetes 集群中运行 PySpark 数据分析任务，数据集为豆瓣电影数据集。附加题部分保留了监控系统、CI/CD 流水线 and K3s + MQTT 边缘计算模拟的实现记录，其中部分云端验证截图由组员闫硕补充完成。由于实验跨越本地、CloudShell、GitHub Actions 和组员补充环境，报告以“任务结果是否满足评分项”为组织原则，并在关键位置给出对应截图或代码文件。

小组分工与环境信息

小组分工

成员	学号	分工比例	主要分工
刘俊豪	2023112456	50%	负责本地容器化、Kubernetes YAML 编写与调整、Spark 数据分析代码、镜像构建推送、主报告撰写与材料整合。
闫硕	2023112447	50%	负责部分云端平台验证、任务 4/5/6 的补充实验截图、监控系统、CI/CD 与边缘计算附加实验截图补充，并协助核对部署结果。

表 1 小组成员与分工说明

实验环境

项目	记录
Region	华东-上海一（cn-east-3）
CCE 集群名称	cloude-ljh-ys-kube
集群类型	CCE Standard
Kubernetes 版本	v1.35.3-r0-35.0.8
Worker 节点数量	基础集群 2 个 Worker；高负载与附加题验证阶段按资源情况使用组员补充环境截图
SWR 组织名	cloud-ljh-ys
方向选择	方向 A：Spark 大数据分析
附加题	监控系统、CI/CD、K3s + MQTT 边缘计算模拟

表 2 实验环境信息汇总

如图 图 1 和图 图 2 所示，CCE 控制台中集群 cloude-ljh-ys-kube 处于运行状态，可用节点数为 2；`kubectl get nodes -o wide` 结果显示两个 Worker 节点均为 Ready，版本为 v1.35.3-r0-35.0.8，满足后续部署和 Spark 作业运行要求。



图 1 CCE 集群概览与 2 个 Worker 节点可用

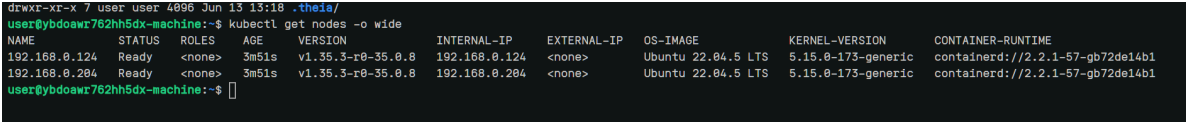


图 2 kubectl get nodes -o wide 输出，两个 Worker 节点均为 Ready

评分项覆盖自查

为避免遗漏，本报告按任务书评分项进行自查。第一部分 1-6 均给出部署步骤、关键 YAML 或命令、截图和问题处理；第二部分方向 A 覆盖 Spark Operator 环境、数据清洗、4 类统计查询和性能分析；附加题覆盖监控系统、CI/CD 流水线和 C-2 边缘计算模拟。各模块证据索引见表 表 3。

模块	评分要求	报告证据
任务 1	Dockerfile 多阶段、自选依赖、前端学号姓名、本地联调、SWR 推送	图 图 1、图 2、图 3、图 4、图 5
任务 2	CCE 集群版本和 2 个 Worker Ready	图 图 1、图 2
任务 3	Deployment、Service、ConfigMap、Secret 与公网 API	图 图 6、图 7、图 9、图 10
任务 4	PVC Bound、写入、删 Pod、重建后读取	图 图 11、图 12、图 13、图 14
任务 5	ConfigMap Volume 挂载和配置更新验证	图 图 15
任务 6	HPA 扩容、停压后回收过程与弹性分析	图 图 16、图 17、图 18
方向 A	Spark 环境、清洗、4 类查询、性能对比	图 图 2 至 图 9，表 表 2、表 3
附加题	监控、CI/CD、前沿专题	图 图 1 至 图 5

表 3 课程设计评分项覆盖自查

第一部分 云计算平台搭建

任务 1 应用容器化

本地应用脚手架位于 scaffold/part1-app/，包含 Flask 后端、Nginx 前端与 Redis。后端程序在 backend/app.py 中提供 /api/ping 与 /api/redis 两个接口，前端页面在 frontend/static/index.html 中嵌入学号与姓名字段，用于验收识别。docker-compose.yml 将 backend 暴露为宿主机 5000，frontend 暴露为宿主机 18080，避免与本机 8080 端口冲突。

在容器化过程中，后端 Dockerfile 保留了 builder 和 runtime 两阶段结构，依赖先安装到 /build/packages，运行时镜像只复制依赖目录和应用代码，从而减少运行镜像中不必要的构建工具。后端镜像额外保留了 requests==2.32.3 依赖；前端首页显示“学号：

2023112456”和“姓名：刘俊豪”。执行 `docker compose up --build` 后，Redis 输出

Ready to accept connections tcp , Nginx 输出

Configuration complete; ready for start up , Flask 输出

Running on http://127.0.0.1:5000 , 说明三容器均已正常拉起，如图 图 1 所示。

```
redis-1 | 1:M 13 Jun 2026 12:26:14.206 * Ready to accept connections tcp
frontend-1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
frontend-1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
frontend-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
frontend-1 | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
frontend-1 | 10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
frontend-1 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
frontend-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
frontend-1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
frontend-1 | /docker-entrypoint.sh: Configuration complete; ready for start up
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: using the "epoll" event method
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: nginx/1.31.1
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:1048576
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker processes
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 29
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 30
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 31
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 32
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 33
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 34
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 35
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 36
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 37
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 38
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 39
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 40
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 41
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 42
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 43
frontend-1 | 2026/06/13 12:26:14 [notice] 1#1: start worker process 44
backend-1 | * Serving Flask app 'app'
backend-1 | * Debug mode: off
backend-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
backend-1 | * Running on all addresses (0.0.0.0)
backend-1 | * Running on http://127.0.0.1:5000
backend-1 | * Running on http://172.19.0.3:5000
backend-1 | Press CTRL+C to quit
(base)
```

图 1 docker compose 本地三容器运行成功

本地联调命令如下：

```
bash
1 cd scaffold/part1-app
2 docker compose up --build
3 curl http://localhost:5000/api/ping
4 curl http://localhost:5000/api/redis
```

执行结果中， `/api/ping` 返回 `{"status":"ok"}` , `/api/redis` 返回 `{"redis":"ok"}` , 表明后端接口正常且 Redis 读写链路可用，如图 图 2 所示。前端页面通过浏览器访问 `http://localhost:18080` 后，页面中正确显示了课程名称、学号、姓名和健康检查入口，如图 图 3 所示。

```
$ curl http://localhost:5000/api/ping
{"status":"ok"}
(base)
# ljh @ DESKTOP-A0DOC7U in ~/project/cloud-compute-design/scaffold/part1-app on git:main x [20:28:54]
$ curl http://localhost:5000/api/redis
{"redis":"ok"}
(base)
```

图 2 本地后端 API 与 Redis 联通验证

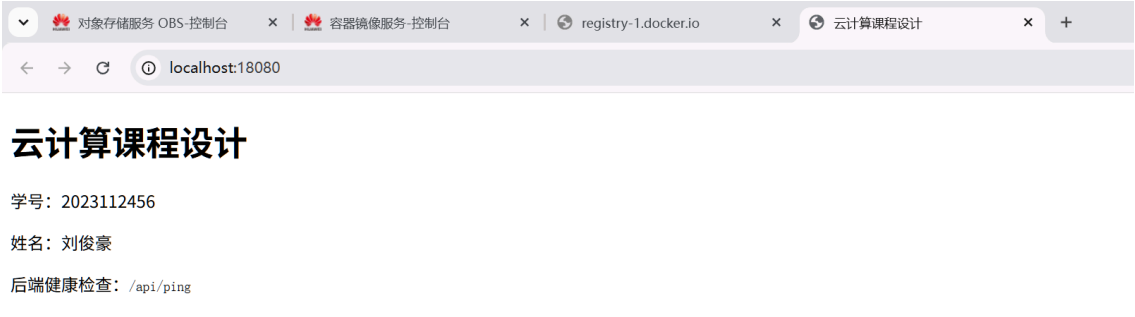


图 3 本地前端页面显示学号姓名

镜像构建完成后，分别推送到 SWR 组织 `cloud-ljh-ys` 下的 `backend:v1` 与 `frontend:v1`。推送日志中可见 `backend` 镜像摘要为 `sha256:5344ae70b9d1f230b7e9461765115fab08ecdaef88ab398803dcd0606e32ae`，`frontend` 镜像摘要为 `sha256:f964948dde26b2ecbe236a96f08780b748e3370bbaa4a9a26f6aef980375f805`。终端推送结果和 SWR 控制台镜像列表分别见图 4 与图 5。

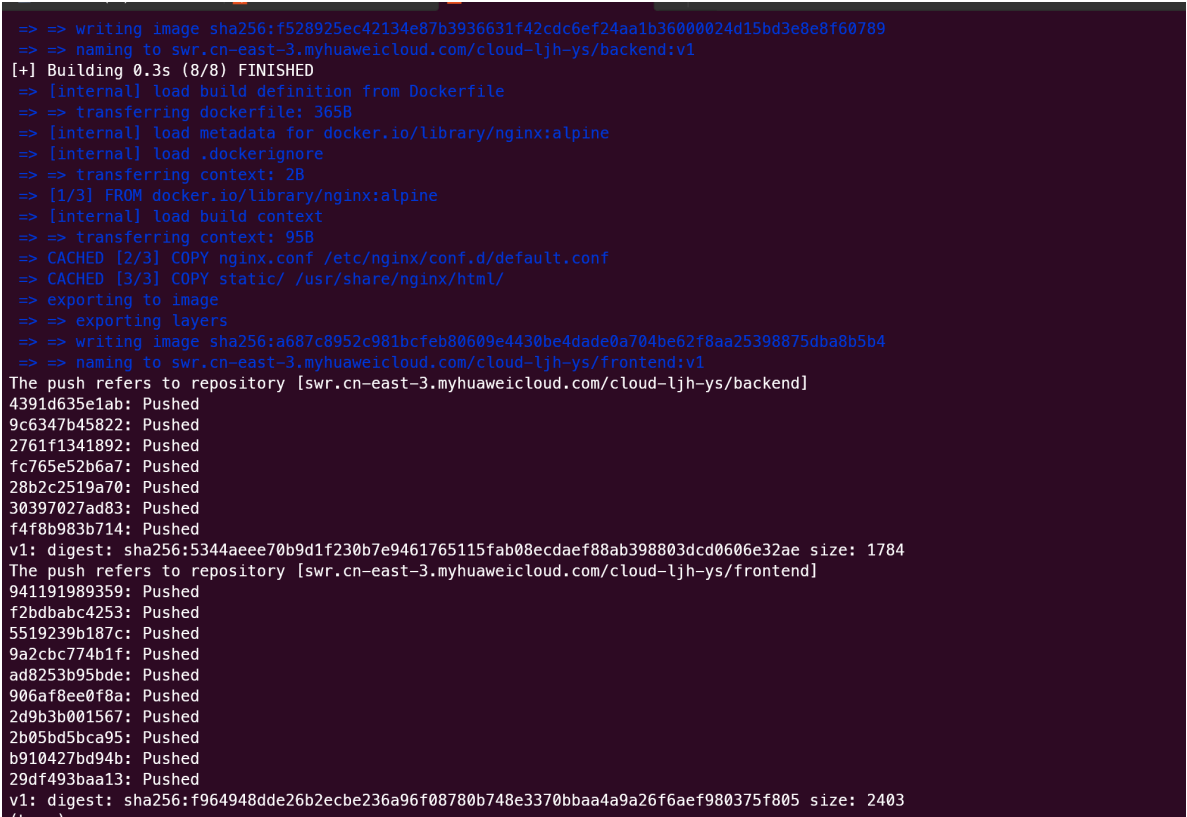


图 4 backend 与 frontend 镜像推送到 SWR

自有镜像

Q

⊗

镜像名称	所属组织	类型	版本号	更新时间	操作
☒ frontend	cloud-ljh-ys	私有	1	2026/06/13 20:42:31 GMT+08:00	镜像自动同步
☒ backend	cloud-ljh-ys	私有	1	2026/06/13 20:42:22 GMT+08:00	镜像自动同步

图 5 SWR 控制台中 backend 与 frontend 镜像列表

本任务中遇到的主要问题是本机端口冲突。早期实验中 frontend 曾尝试绑定宿主机 8080 端口，但该端口已被本机其他程序占用，导致 Docker 报错 `failed to bind host port 0.0.0.0:8080/tcp: address already in use`。因此最终将前端映射端口调整为 18080:80，使本地调试顺利完成。

任务 2 CCE 集群搭建

本组在华为云 CCE 中创建 Standard 集群 `cloude-ljh-ys-kube`，区域与镜像仓库保持一致，均为 `cn-east-3`。控制台显示集群版本为 `v1.35`，可用节点数为 `2 / 2`，表明底层工作节点资源准备完成。使用 CloudShell 执行 `kubectl get nodes -o wide` 后，可见两个工作节点的状态均为 `Ready`，内网地址分别为 `192.168.0.124` 与 `192.168.0.204`。

任务 2 的关键结果已经在图 图 1 与图 图 2 中给出。该步骤为后续部署 Deployment、Service、PVC、Spark Operator 等资源提供了稳定的 Kubernetes 运行环境。

任务 3 应用部署

任务 3 所使用的 Kubernetes 清单位于 `scaffold/part1-k8s/`。其中：

- `backend-deployment.yaml` 将后端副本数设置为 2，并通过 `envFrom` 注入 `backend-config`，再通过 `secretKeyRef` 注入 Redis 密码。
- `redis-deployment.yaml` 将 Redis 副本数设置为 1，启用 `appendonly yes` 并使用 Secret 中的密码。
- `service.yaml` 中 `backend-svc` 为 `LoadBalancer` 类型，包含 `kubernetes.io/elb.class: union` 注解；`redis-svc` 为 `ClusterIP`，仅在集群内部暴露。
- `configmap-secret.yaml` 提供 `REDIS_HOST=redis-svc`、`REDIS_PORT=6379` 以及 Redis 密码的 base64 编码结果。

部署完成后，`kubectl get pod -o wide` 显示 backend 两个副本、frontend 一个副本和 redis 一个副本均进入 `Running` 状态，如图 图 6 所示。随后查看 `kubectl get svc -o wide` 和 `kubectl get pvc`，可见 `backend-svc` 初始为 `LoadBalancer`、`redis-svc` 为 `ClusterIP`，`redis-data-pvc` 已成功 `Bound`，如图 图 7 所示。

```
user@ybdoawr762hh5dx-machine:~/projects/cloud-compute-ex$ kubectl get pod -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
backend-5f6c8cb5d8-56x9q            1/1     Running   0           12m   10.0.0.133      192.168.0.124    <none>            <none>
backend-5f6c8cb5d8-87ldb            1/1     Running   0           12m   10.0.1.130      192.168.0.219    <none>            <none>
frontend-b586bbb4f-gr2zj           1/1     Running   0           12m   10.0.1.6        192.168.0.233    <none>            <none>
redis-b9bc4658f-cx1ql              1/1     Running   0           12m   10.0.0.5        192.168.0.204    <none>            <none>
user@ybdoawr762hh5dx-machine:~/projects/cloud-compute-ex$
```

图 6 任务 3 中 backend、frontend、redis Pod 全部 Running

```
user@ybdoawr762hh5dx-machine:~/projects/cloud-compute-ex$ kubectl get svc -o wide
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE   SELECTOR
backend-svc     LoadBalancer  10.247.6.149   <pending>      80:31377/TCP     24m   app=backend
kubernetes     ClusterIP    10.247.0.1     <none>         443/TCP          65m   <none>
redis-svc       ClusterIP    10.247.226.41 <none>         6379/TCP         24m   app=redis
user@ybdoawr762hh5dx-machine:~/projects/cloud-compute-ex$ kubectl get pvc
NAME            STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
redis-data-pvc  Bound    pvc-6206d8e6-f706-4e4c-ac72-bf942ccaf080  10Gi       RWO             csi-disk       <unset>                24m
user@ybdoawr762hh5dx-machine:~/projects/cloud-compute-ex$
```

图 7 Service 与 PVC 状态检查

CCE 中的后端服务后续成功绑定到 ELB。控制台中创建的负载均衡器名称为 `elb-b373`，ID 为 `60ea6c69-7fc8-41d0-99e2-44e7343820f9`，公网地址对应 `119.3.155.15`。图 图 8 给出了 ELB 控制台结果，图 图 9 给出了 `backend-svc` 绑定 ELB 后的 `kubectl get svc` 结果。

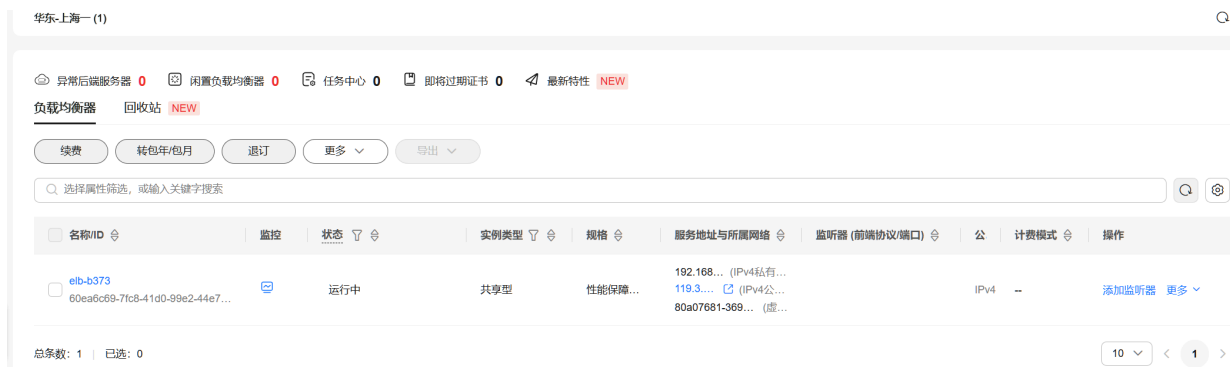


图 8 华为云 ELB 创建结果

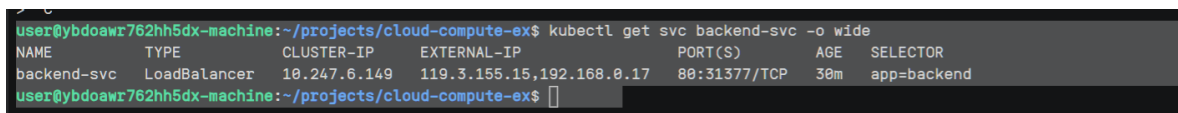


图 9 backend-svc 绑定 ELB 后的 Service 信息

通过浏览器或 `curl` 访问公网入口后，`/api/ping` 返回 `{"status":"ok"}`，`/api/redis` 返回 `{"redis":"ok"}`，证明从公网入口到 Service、Pod、Redis 的完整访问链路工作正常，如图 图 10 所示。

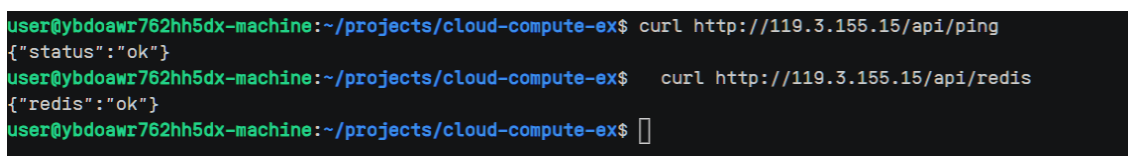


图 10 公网访问 backend API 成功

从资源编排的角度看，任务 3 完成了“本地容器镜像 -> 私有镜像仓库 -> Kubernetes Deployment/Service -> 华为云 ELB 暴露”的完整链路，这一步也是第一部分后续功能验证的基础。

任务 4 Redis 持久化存储

任务 4 的目标是验证 Redis Pod 删除后数据不会丢失。该任务使用 `redis-pvc.yaml` 创建了 `redis-data-pvc`，存储类为 `csi-disk`，挂载路径为 Redis 容器内的 `/data`。PVC 绑定成功后，首先执行 `kubectl get pvc` 验证其状态为 `Bound`，结果如图 图 11 所示。

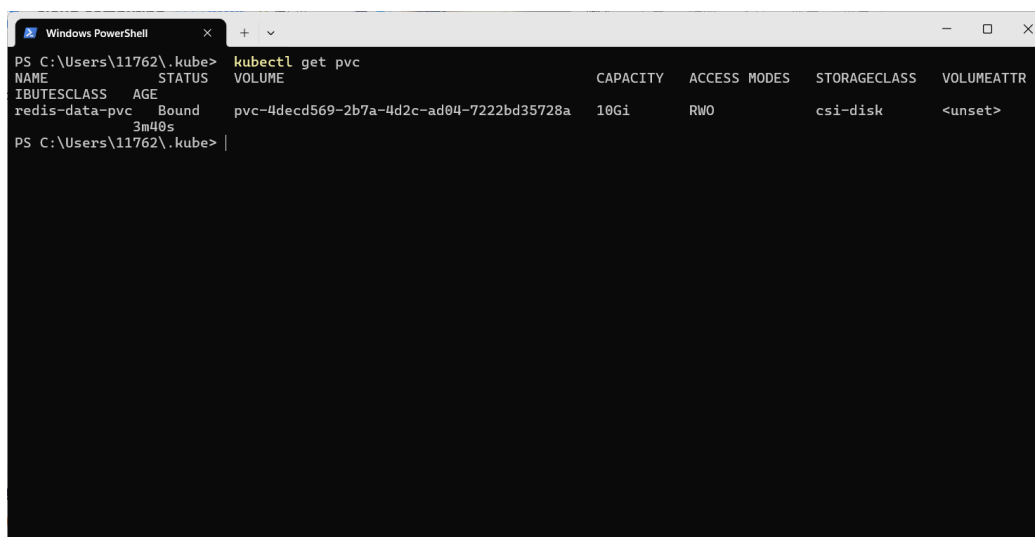


图 11 Redis PVC 绑定成功

随后使用如下命令向 Redis 写入测试数据：

```
bash
1 kubectl exec -it $(kubectl get pod -l app=redis -o
  jsonpath='{.items[0].metadata.name}') -- \
2 redis-cli SET testkey "hello"
```

返回结果为 OK，如图 图 12 所示。之后删除当前 Redis Pod，使 Deployment 自动重建：

```
bash
1 kubectl delete pod $(kubectl get pod -l app=redis -o
  jsonpath='{.items[0].metadata.name}')
```

Pod 删除成功的结果如图 图 13 所示。待新 Pod 重新启动后，再次执行 GET testkey，返回 "hello"，说明数据未随 Pod 生命周期一起丢失，如图 图 14 所示。

```
Windows PowerShell
PS C:\Users\11762\.kube> kubectl get pvc
NAME                                STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTR
redis-data-pvc                      Bound    pvc-4decd569-2b7a-4d2c-ad04-7222bd35728a  10Gi       RWO            csi-disk       <unset>
PS C:\Users\11762\.kube> kubectl exec -it $(kubectl get pod -l app=redis -o jsonpath='{.items[0].metadata.name}') -- re
dis-cli SET testkey "hello"
OK
PS C:\Users\11762\.kube> |
```

图 12 向 Redis 写入测试数据 testkey=hello

```
Windows PowerShell
PS C:\Users\11762\.kube> kubectl get pvc
NAME                                STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTR
redis-data-pvc                      Bound    pvc-4decd569-2b7a-4d2c-ad04-7222bd35728a  10Gi       RWO            csi-disk       <unset>
PS C:\Users\11762\.kube> kubectl exec -it $(kubectl get pod -l app=redis -o jsonpath='{.items[0].metadata.name}') -- re
dis-cli SET testkey "hello"
OK
PS C:\Users\11762\.kube> kubectl delete pod $(kubectl get pod -l app=redis -o jsonpath='{.items[0].metadata.name}')
pod "redis-8647cdcc58-stml2" deleted from default namespace
PS C:\Users\11762\.kube> |
```

图 13 删除 Redis Pod 触发自动重建

```

PS C:\Users\11762\.kube> kubectl get pvc
NAME                                STATUS    CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTR
redis-data-pvc                     Bound    10Gi       RWO            csi-disk       <unset>
PS C:\Users\11762\.kube> kubectl exec -it $(kubectl get pod -l app=redis -o jsonpath='{.items[0].metadata.name}') -- redis-cli SET testkey "hello"
OK
PS C:\Users\11762\.kube> kubectl delete pod $(kubectl get pod -l app=redis -o jsonpath='{.items[0].metadata.name}')
pod "redis-8647cdcc58-stm12" deleted from default namespace
PS C:\Users\11762\.kube> kubectl exec -it $(kubectl get pod -l app=redis -o jsonpath='{.items[0].metadata.name}') -- redis-cli GET testkey
"hello"
PS C:\Users\11762\.kube>

```

图 14 Redis Pod 重建后仍可读回 hello

该实验说明云硬盘 PVC 真正承载了 Redis 的持久化数据文件。即使容器实例销毁并重建，只要 Deployment 仍然挂载同一块 PVC，Redis 的关键数据就可以保留下来。对于云原生应用而言，这一步体现了“无状态计算实例”和“有状态数据存储”之间的职责分离。

任务 5 ConfigMap Volume 挂载

任务 5 将前端 Nginx 的反向代理配置改为 ConfigMap Volume 挂载方式。清单位于 `frontend-nginx-configmap.yaml` 和 `frontend-deployment.yaml` 中，其中 `default.conf` 被作为单文件挂载到 `/etc/nginx/conf.d/default.conf`。组员闫硕补充完成的截图表明，第一次查看容器内配置文件时，`proxy_pass` 指向 `http://backend-svc:5000`；修改 ConfigMap 后再次执行 `cat /etc/nginx/conf.d/default.conf`，文件中的 `proxy_pass` 已变为 `http://backend-svc:5001`，如图 15 所示。

```

PS C:\Users\11762\.kube> kubectl exec -it $(kubectl get pod -l app=frontend -o jsonpath='{.items[0].metadata.name}') -- cat /etc/nginx/conf.d/default.conf
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://backend-svc:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
PS C:\Users\11762\.kube> kubectl edit configmap nginx-config
configmap/nginx-config edited
PS C:\Users\11762\.kube> sleep 30
PS C:\Users\11762\.kube> kubectl exec -it $(kubectl get pod -l app=frontend -o jsonpath='{.items[0].metadata.name}') -- cat /etc/nginx/conf.d/default.conf
server {
    listen 80;
    server_name localhost;

    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://backend-svc:5001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
PS C:\Users\11762\.kube>

```

图 15 ConfigMap Volume 挂载与配置热更新验证

从实验现象可以看出，ConfigMap Volume 方式更适合承载整段配置文件，例如 Nginx 的 `default.conf`、应用的 YAML 配置、脚本模板等；而 `envFrom` 更适合注入离散的关键值

对环境变量，例如 Redis 主机名、端口号、运行模式等。二者的适用边界在于“配置是否需要以文件结构存在”：若应用本身直接读取环境变量，则 `envFrom` 简单直接；若应用只能读取磁盘配置文件，则应使用 `ConfigMap Volume`。

任务 6 HPA 弹性伸缩

任务 6 使用 `hpa.yaml` 为 backend Deployment 配置了 `autoscaling/v2` 版本的 HPA，对象名为 `backend-hpa`，最小副本数 1、最大副本数 4、CPU 平均利用率目标 60%。组员补充截图表明，在压测开始前，`kubectl get pods -w` 中 backend 仅有 1 个副本，frontend 与 redis 也均处于 `Running` 状态，如图 图 16 所示。

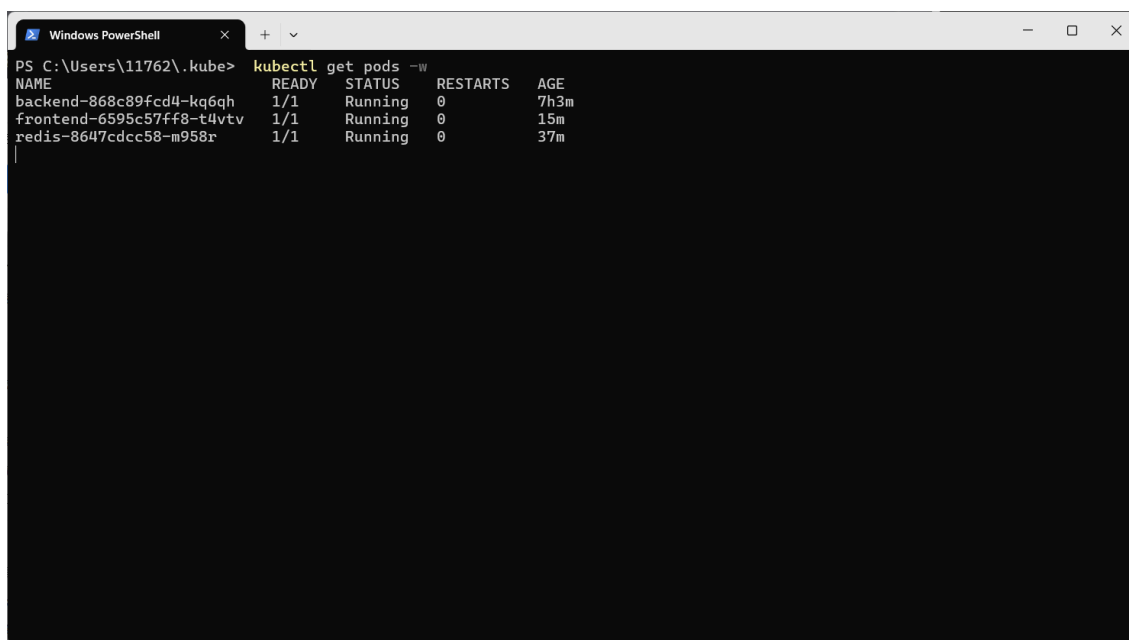


图 16 压测开始前 Pod 状态

随后在外部持续施加请求压力，`kubectl get pods -w` 输出中出现新的 backend Pod，从 `Pending` 到 `ContainerCreating` 再到 `Running`，说明 HPA 已经检测到 CPU 压力并触发扩容，如图 图 17 所示。停压之后，原先被扩出的副本进入 `Terminating` 和 `Error` 回收阶段，如图 图 18 表明系统已经开始缩容回退。

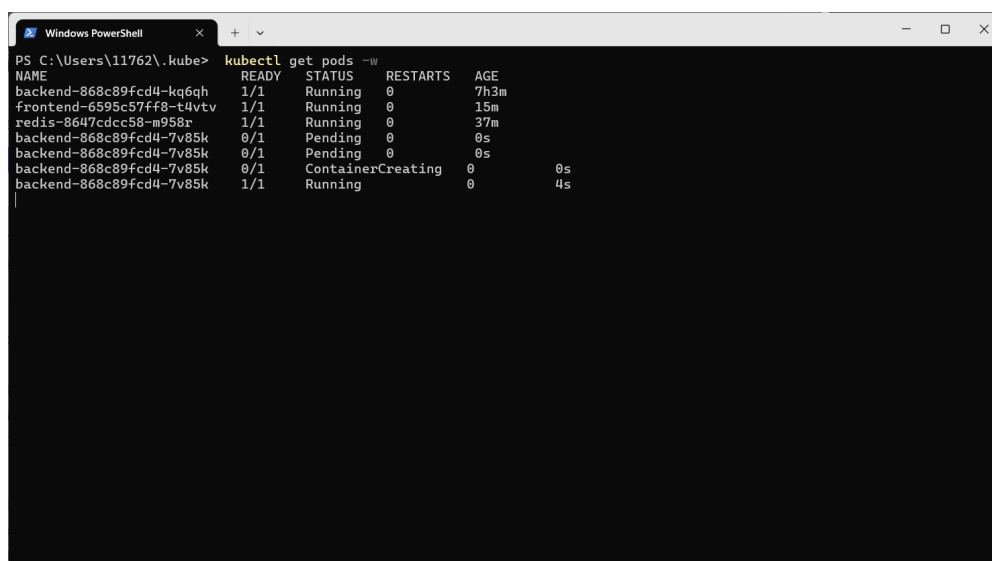


图 17 压测过程中 backend 副本扩容

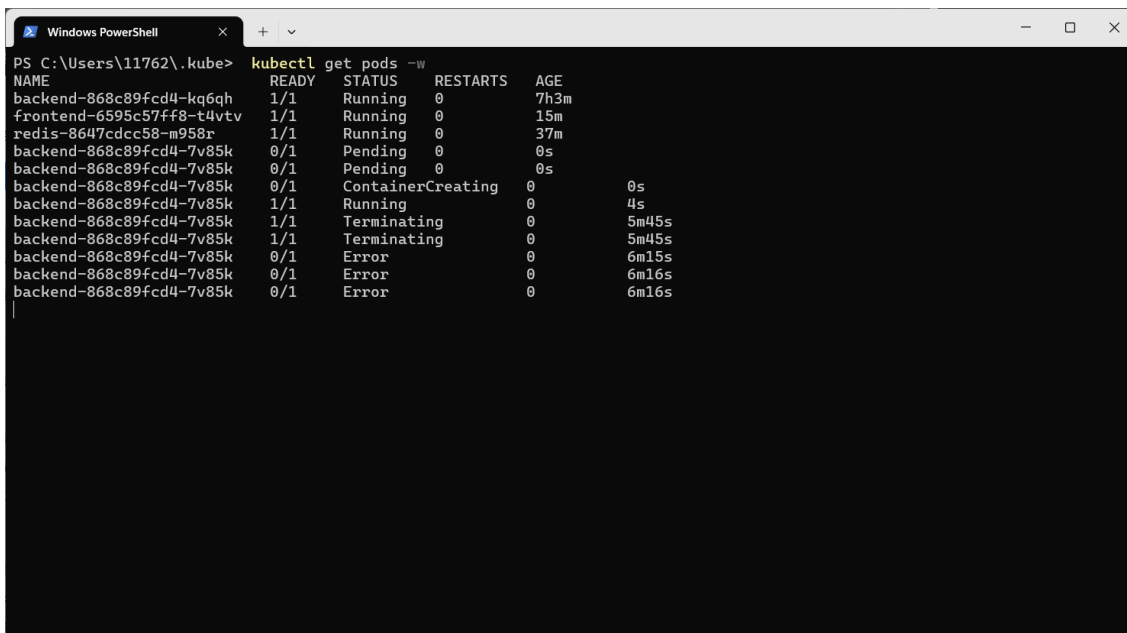


图 18 停压后 backend 副本进入回收过程

HPA 的行为并非瞬时完成，其扩容存在两个主要延迟来源。第一，metrics-server 需要以固定周期采集各 Pod 的资源使用情况；第二，HPA 控制器也会按固定周期重新计算目标副本数，因此不会对瞬时波动做出过于激进的响应。缩容通常比扩容更保守，这是因为 Kubernetes 会设置冷却与稳定窗口，防止副本数在短时间内反复增减，造成抖动和资源浪费。从云平台成本角度看，HPA 可以在低负载时自动缩减副本，在高负载时自动扩容，是实现“按需供给、弹性计费”的直接手段。

第二部分 Spark 大数据分析

A-0 Spark Operator 环境部署

本组第二部分选择 Spark 方向。首先将离线 Spark Operator 镜像和教师提供的 PySpark 基础镜像重新打 tag 并推送至 SWR。镜像推送记录表明：

- spark-operator:2.5.0 的摘要为
sha256:bf10fb90ca4c62f6df55585edd910c05a3973cf7ce38e2bf5adaefd776049294 ；
- pyspark:v9 的摘要为
sha256:d7f87c0b8f3c47110e65b343cc8528b87a48e8a3d513cbf89c68d9256a51039b 。

对应截图见图 图 1。随后使用离线 Helm Chart 执行

helm upgrade --install spark-op ...，成功在 spark-operator 命名空间下部署 Spark Operator，控制器与 webhook 最终均进入 Running 状态，如图 图 2 所示。

```

$ docker push swr.cn-east-3.myhuaweicloud.com/cloud-ljh-ys/spark-operator:2.5.0
The push refers to repository [swr.cn-east-3.myhuaweicloud.com/cloud-ljh-ys/spark-operator]
69a3fe2179ee: Pushed
66959451ca09: Pushed
c98ac252dca3: Pushed
5f70bf18a086: Pushed
947ac208aa80: Pushed
a37ac52c7fdb: Pushed
828ffb1ebd7: Pushed
eb7b2f0ff5f2: Pushed
cb066cb54ea3: Pushed
70ef899564ac: Pushed
44811e11756a: Pushed
aeaad1f66245: Pushed
c730154d906e: Pushed
fd7b85c1b84e: Pushed
2.5.0: digest: sha256:bf10fb90ca4c62f6df55585edd910c05a3973cf7ce38e2bf5adaefd776049294 size: 3462
(base)
# ljh @ DESKTOP-A0D0C7U in ~/project/cloud-compute-design on git:main x [23:03:27]
$ docker push swr.cn-east-3.myhuaweicloud.com/cloud-ljh-ys/pyspark:v9
The push refers to repository [swr.cn-east-3.myhuaweicloud.com/cloud-ljh-ys/pyspark]
0d85f130cca4: Pushed
8c5359e5899b: Pushed
489ff4336d34: Pushed
7e110a8aaaba: Pushed
673ee790f8c6: Pushed
25a12baa36cf: Pushed
e21ac3f5cca3: Pushed
5f70bf18a086: Layer already exists
801f91aa5252: Pushed
dc5ea3876c37: Pushed
1299bc68a3fc: Pushed
48828c7acf9a: Pushed
0dac74c5cbaa: Pushed
4a5ab425d6e1: Pushed
2f30f6654b4e: Pushed
8cdab93a842f: Pushed
3ec3ded77c0c: Pushed
v9: digest: sha256:d7f87c0b8f3c47110e65b343cc8528b87a48e8a3d513cbf89c68d9256a51039b size: 3879

```

图 1 Spark Operator 与 PySpark 镜像推送到 SWR

```

user@bnk9wuvevcp6zpz-machine:~/projects/cloud-compute-ex$ helm upgrade --install spark-op \
> resources/离线包/spark/spark-operator \
> -n spark-operator --create-namespace \
> --set controller.image.repository=swr.cn-east-3.myhuaweicloud.com/cloud-ljh-ys/spark-operator \
> --set controller.image.tag=2.5.0 \
> --set webhook.image.repository=swr.cn-east-3.myhuaweicloud.com/cloud-ljh-ys/spark-operator \
> --set webhook.image.tag=2.5.0
Release "spark-op" does not exist. Installing it now.
NAME: spark-op
LAST DEPLOYED: Sat Jun 13 15:28:36 2026
NAMESPACE: spark-operator
STATUS: deployed
REVISION: 1
TEST SUITE: None

```

图 2 Spark Operator 安装成功

分析任务镜像基于 `scaffold/part2-spark/Dockerfile` 构建，最终镜像地址为 `swr.cn-east-3.myhuaweicloud.com/cloud-ljh-ys/pyspark-analysis:v3`。该镜像将 `analysis.py`、`wordcount.py` 与 `douban_movies.csv` 直接打包到容器内，避免运行时依赖 OBS 访问凭证。提交 `SparkApplication` 后，`driver` 和 `executor` 都经历了 `Pending -> ContainerCreating -> Running -> Completed` 的生命周期，作业整体状态为 `COMPLETED`，如图 3 所示。

```

user@bnk9wuwevcp6zp8-machine:~/projects/cloud-compute-ex$ kubectl get pods -w
NAME                                READY   STATUS             RESTARTS   AGE
pyspark-analysis-driver             0/1     Pending            0           0s
pyspark-analysis-driver             0/1     Pending            0           0s
pyspark-analysis-driver             0/1     ContainerCreating  0           0s
pyspark-analysis-driver             1/1     Running            0           2s
doubanmovieanalysis-d5266f9ec1c8c805-exec-1  0/1     Pending            0           0s
doubanmovieanalysis-d5266f9ec1c8c805-exec-1  0/1     Pending            0           0s
doubanmovieanalysis-d5266f9ec1c8c805-exec-1  0/1     ContainerCreating  0           0s
doubanmovieanalysis-d5266f9ec1c8c805-exec-1  1/1     Running            0           1s
doubanmovieanalysis-d5266f9ec1c8c805-exec-1  1/1     Terminating      0           11s
doubanmovieanalysis-d5266f9ec1c8c805-exec-1  1/1     Terminating      0           11s
doubanmovieanalysis-d5266f9ec1c8c805-exec-1  0/1     Completed          0           12s
doubanmovieanalysis-d5266f9ec1c8c805-exec-1  0/1     Completed          0           12s
doubanmovieanalysis-d5266f9ec1c8c805-exec-1  0/1     Completed          0           12s
pyspark-analysis-driver             0/1     Completed          0           19s
pyspark-analysis-driver             0/1     Completed          0           20s

```

图 3 SparkApplication 提交并成功运行

实际提交的 SparkApplication 参数见表 1。任务书模板建议 `executorInstances=2`、`executorMemory="1g"`；本组在资源受限的 CCE 环境中按实验现象进行了等价降配，将 driver 与 executor 内存设置为 512m，同时通过 `coreRequest: "100m"` 降低 Kubernetes 调度请求。该调整没有改变 Spark 逻辑核心数和 executor 数量，但避免了小规格节点因请求资源过高而无法调度。

参数	实际取值	作用说明
apiVersion / kind	sparkoperator.k8s.io/ v1beta2 SparkApplication	使用 Spark Operator 的 CRD 提交 PySpark 作业。
mode / type	cluster / Python	driver 在 Kubernetes Pod 中运行，主程序为 Python 脚本。
image	swr.cn-east-3.myhuaweicloud.com/ cloud-ljh-ys/pyspark-analysis:v3	使用推送到 SWR 的分析镜像。
mainApplicationFile	local:///opt/spark/work/ analysis.py	容器内直接读取分析脚本，避免运行时远程拉取。
driver	cores=1, coreRequest=100m, memory=512m, memoryOverhead=128m	保证 driver 可在小规格节点中调度。
executor	instances=2, cores=1, coreRequest=100m, memory=512m, memoryOverhead=128m	满足 2 个 executor 的并行实验要求，并降低调度压力。
restartPolicy	Never	作业型任务失败后不自动反复重启，便于查看日志定位。
serviceAccount	spark-op-spark-operator-spark	授予 driver 创建 executor Pod 所需权限。

表 1 SparkApplication 关键参数说明

初次提交 Spark 作业时，driver 因资源申请过高而出现 `FailedScheduling`。后续将 driver memory 调整为 512m、为 driver/executor 增加 `coreRequest: "100m"` 与 `memoryOverhead: "128m"`，并采用将数据文件直接打包入镜像的方式规避了 `s3a://` 凭证缺失问题，最终使任务顺利完成。

A-1 数据清洗

数据集采用 `asst/douban_movies.csv`。`analysis.py` 中首先读取 CSV，随后将 `year`、`rating_score`、`rating_count`、`collect_count` 转换为数值型字段；在清洗阶段，对 `year` 和 `rating_score` 使用 `dropna`，对 `genres`、`countries`、`directors`、`summary`、`rating_count` 和 `collect_count` 使用 `fillna`。

作业日志中包含数据读取、清洗和统计输出，清洗作业完成证据见图 4。字段缺失比例统计结果如下表所示。

```
26/06/13 16:27:39 INFO TaskSetManager: Starting task 0.0 in stage 51.0 (TID 31) (10.0.1.16, executor 1, partition 0, NODE_LOCAL, 73
26/06/13 16:27:39 INFO BlockManagerInfo: Added broadcast_46_piece0 in memory on 10.0.1.16:37981 (size: 21.4 KiB, free: 116.8 MiB)
26/06/13 16:27:39 INFO MapOutputTrackerMasterEndpoint: Asked to send map output locations for shuffle 13 to 10.0.1.16:44954
26/06/13 16:27:39 INFO TaskSetManager: Finished task 0.0 in stage 51.0 (TID 31) in 150 ms on 10.0.1.16 (executor 1) (1/1)
26/06/13 16:27:39 INFO DAGScheduler: ResultStage 51 (showString at <unknown>:0) finished in 0.156 s
26/06/13 16:27:39 INFO DAGScheduler: Job 31 is finished. Cancelling potential speculative or zombie tasks for this job
26/06/13 16:27:39 INFO TaskSchedulerImpl: Removed TaskSet 51.0, whose tasks have all completed, from pool
26/06/13 16:27:39 INFO TaskSchedulerImpl: Killing all running tasks in stage 51: Stage finished
26/06/13 16:27:39 INFO DAGScheduler: Job 31 finished: showString at <unknown>:0, took 0.159063 s
26/06/13 16:27:39 INFO CodeGenerator: Code generated in 4.729686 ms
26/06/13 16:27:39 INFO CodeGenerator: Code generated in 3.867428 ms

+-----+
|country|rank_in_country|title|year|rating_score|
+-----+
| |1|天蝎座符号|1977.0|0.0|
|1958-06-29|1|小小白色女裙|1958.0|8.1|
|Antigua and Barbuda|1|Memoirs of the Blue|2010.0|0.0|
|Armenia|1|On the Old Roman Road|2001.0|0.0|
|Aruba|1|无间交易|2004.0|5.6|
|Bahamas|1|足球野小子4|2007.0|0.0|
|Belarus|1|Рейнджер из атомной зоны|1999.0|0.0|
|Bill Melendez|1|Charlie Brown"|0.0|27.0|
|Burkina Faso|1|基尼与亚当斯|1997.0|0.0|
|Burkina Faso|2|DeLwende|2005.0|0.0|
|Cambodia|1|战后的一夜|1998.0|0.0|
|Cameroon|1|Clando|1996.0|0.0|
|Canada 加拿大|1|Reluctant Witness|2014.0|0.0|
|Cyprus|1|在星空下|2001.0|7.3|
|Cyprus"|1|Dyo ilioi ston ourano|1991.0|0.0|
|Dominica|1|Memoirs of the Blue|2010.0|0.0|
|Ecuador|1|沼泽的乌鸦|1974.0|0.0|
|Ethiopia|1|阿杜瓦|1999.0|0.0|
|Fiji|1|修理工|2012.0|0.0|
|Gallifrey|1|博士的女伴|2012.0|9.1|
+-----+
only showing top 20 rows

26/06/13 16:27:39 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/06/13 16:27:39 INFO SparkUI: Stopped Spark web UI at http://pyspark-analysis-bbd79f9ec1cef83d-driver-svc.default.svc:4040
26/06/13 16:27:39 INFO KubernetesClusterSchedulerBackend: Shutting down all executors
26/06/13 16:27:39 INFO KubernetesClusterSchedulerBackend$KubernetesDriverEndpoint: Asking each executor to shut down
26/06/13 16:27:39 WARN ExecutorPodsWatchSnapshotSource: Kubernetes client has been closed.
26/06/13 16:27:39 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/06/13 16:27:39 INFO MemoryStore: MemoryStore cleared
```

图 4 A-1 数据清洗作业日志与完成输出

字段	缺失比例	处理策略	说明
year	7.24%	dropna	年份是时间趋势分析的核心字段，缺失会直接影响时间维度统计。
rating_score	9.15%	dropna	评分是排序和聚合分析的核心字段，缺失样本不宜继续参与统计。
genres	11.77%	fillna(“未知”)	文本类字段缺失不应轻易丢弃整行记录。
countries	4.17%	fillna(“未知”)	保留记录，用于后续国家/地区维度的窗口分析。
directors	16.75%	fillna(“未知”)	导演信息缺失不影响其他核心统计。
summary	31.36%	fillna(“暂无简介”)	简介缺失比例很高，但不是主要分析依据。

表 2 豆瓣电影数据集主要缺失字段处理策略

清洗前数据总量为 70189 行，清洗后剩余 61841 行，约减少 11.9%。这一变化说明删除策略主要作用在缺失关键数值字段的记录上，而对文本类字段则以保留样本为主。其优点在于既保证核心统计字段的可靠性，又避免因非关键字段缺失造成样本规模断崖式下降。

数值字段的统计摘要可概括为：`year` 平均值约为 1998.26，标准差为 22.00；`rating_count` 平均值约为 4784.97；`collect_count` 平均值约为 7405.81。从结果看，数据集主体分布在现代电影时间区间内，同时存在较高的人气离散性，适合后续做聚合、Top-N 与趋势分析。

A-2 Spark SQL 统计分析

本组在 `analysis.py` 中实现了 4 类查询：GROUP BY 聚合、Top-N 排序、按年份趋势分析和窗口函数分组排名。各查询结果与分析如下。

Q1 按类型聚合统计

Q1 通过 `explode + groupBy` 将复合类型字段拆分为独立类型，然后统计各类型电影数量、平均评分和平均评分人数。查询结果如图 5 所示。

```
=== Q1_GROUP_BY_genre (1.851s) ===
```

genre	movie_count	avg_rating	avg_rating_count
剧情	28087	2.83	6247.0
喜剧	13402	2.5	5445.0
动作	8701	2.21	6785.0
爱情	8121	2.89	7898.0
恐怖	7673	1.48	1202.0
惊悚	6685	2.38	4081.0
犯罪	6027	2.44	7012.0
未知	4964	2.54	5030.0
悬疑	3994	2.52	7893.0
动画	3244	3.42	9084.0
科幻	2989	2.27	9333.0
冒险	2539	3.01	13803.0
奇幻	1946	3.21	15779.0
家庭	1583	3.21	8577.0
音乐	1444	2.66	3639.0
歌舞	1176	1.83	1683.0
战争	1103	3.48	6347.0
传记	1101	3.75	12272.0
历史	950	3.91	9740.0
同性	757	4.18	7247.0

only showing top 20 rows

图 5 Q1：按电影类型统计数量、平均评分和平均评分人数

从结果可以看出，**剧情**、**喜剧**、**动作**、**爱情** 构成了样本主体，其中 **剧情** 类型达到 28087 部，数量远高于其他类别，说明豆瓣电影数据集中剧情片占比极高。与此同时，**同性**、**历史**、**传记**、**战争** 等类型虽然数量不大，但平均评分明显更高，这表明小众类型在样本中呈现出“数量少但口碑较高”的长尾现象。**动画**、**奇幻** 和 **冒险** 的平均评分人数也较高，说明这些类型更容易形成大众关注度。

Q2 收藏数 Top-N

Q2 直接按照 `collect_count` 进行降序排序并取前 10 名，结果如图 图 6 所示。

```
=== Q2_TOP_N_collect_count (0.571s) ===
```

title	year	rating_score	collect_count
我不是药神	2018.0	9.0	4235488.0
肖申克的救赎	1994.0	9.7	2866850.0
这个杀手不太冷	1994.0	9.4	2788701.0
阿甘正传	1994.0	9.5	2435797.0
千与千寻	2001.0	9.4	2433371.0
泰坦尼克号	1997.0	9.4	2406355.0
盗梦空间	2010.0	9.3	2328477.0
霸王别姬	1993.0	9.6	2266871.0
三傻大闹宝莱坞	2009.0	9.2	2099002.0
摔跤吧！爸爸	2016.0	9.0	2057973.0

图 6 Q2: 按收藏数排序得到 Top 10 电影

榜单中排名靠前的电影包括《我不是药神》《肖申克的救赎》《这个杀手不太冷》《阿甘正传》《千与千寻》等，评分基本都维持在 9 分上下。这说明在当前数据集中，收藏量和评分之间存在明显的正相关关系，即高收藏往往对应高口碑。另一方面，榜单涵盖中国、美国、日本、印度等多个国家或地区的经典作品，也说明该数据集的文化来源较为多元。

Q3 按年份趋势分析

Q3 对 `year` 做整型转换后，统计每年的电影数量、平均评分和总评分人数，结果如图 图 7 所示。

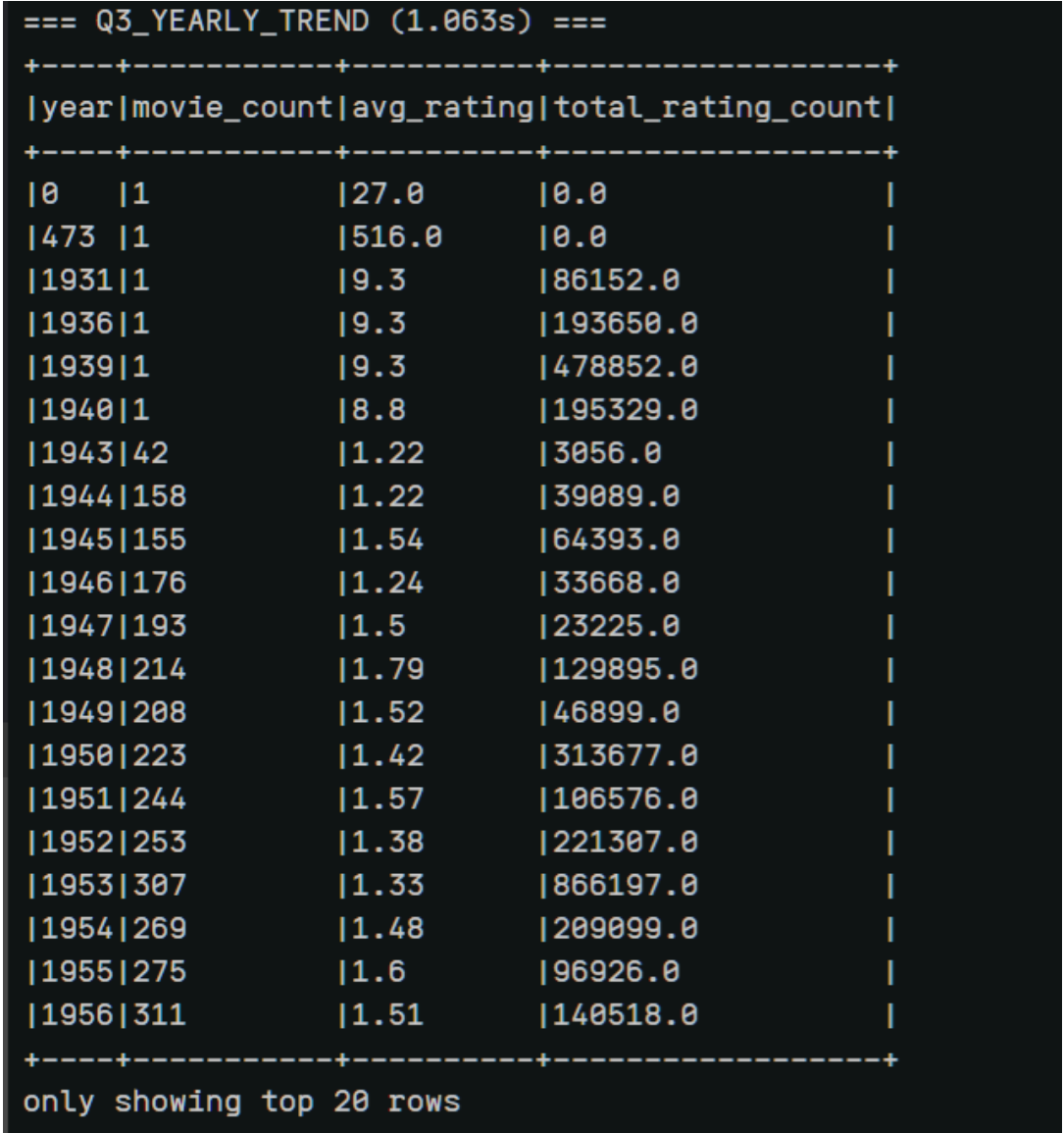


图 7 Q3: 按年份统计电影数量、平均评分和总评分人数

结果中出现了 0 年和 473 年等明显异常值，说明原始数据中的年份字段仍然存在脏值。虽然这些脏值不会阻止作业运行，但会影响时间趋势图的可信度，因此若后续需要更严格的趋势研究，应额外加入年份区间过滤，例如限制在 1900 <= year <= 2025。除异常值外，1940 年代之后的样本数量开始持续增加，说明数据集的主体电影记录仍集中在现代电影时期。

Q4 窗口函数分组排名

Q4 使用

```
row_number over(partition by country order by rating_score desc, rating_count desc)
```

对不

同国家或地区内的电影进行评分排名，结果如图 图 8 所示。



图 8 Q4：按国家或地区分组后提取评分前三电影

该查询验证了窗口函数在分组内排序问题中的有效性，但也暴露出 `countries` 列存在进一步清洗需求。例如输出中出现了 "、1958-06-29、Bill Melendez 这类明显不是国家名称的值，说明原始字段中混入了日期、人物名和异常分隔字符串。换言之，窗口函数本身没有问题，问题在于国家字段的源数据还需要做更细的标准化处理，这一点也体现了数据工程中“清洗质量决定分析质量”的规律。

A-3 性能对比与 Amdahl 分析

本组选择 Q1 作为性能对比对象，分别使用 Pandas 单机实现和 PySpark 分布式实现进行测试。根据本地与 Spark driver 输出，三组结果如下表所示。

实现方式	Executor 数	运行时间 / s	相对 Pandas 加速比
Pandas	0	0.954	1.00
PySpark	1	1.751	0.55
PySpark	2	1.851	0.52

表 3 Pandas 与 PySpark 的 Q1 查询耗时对比

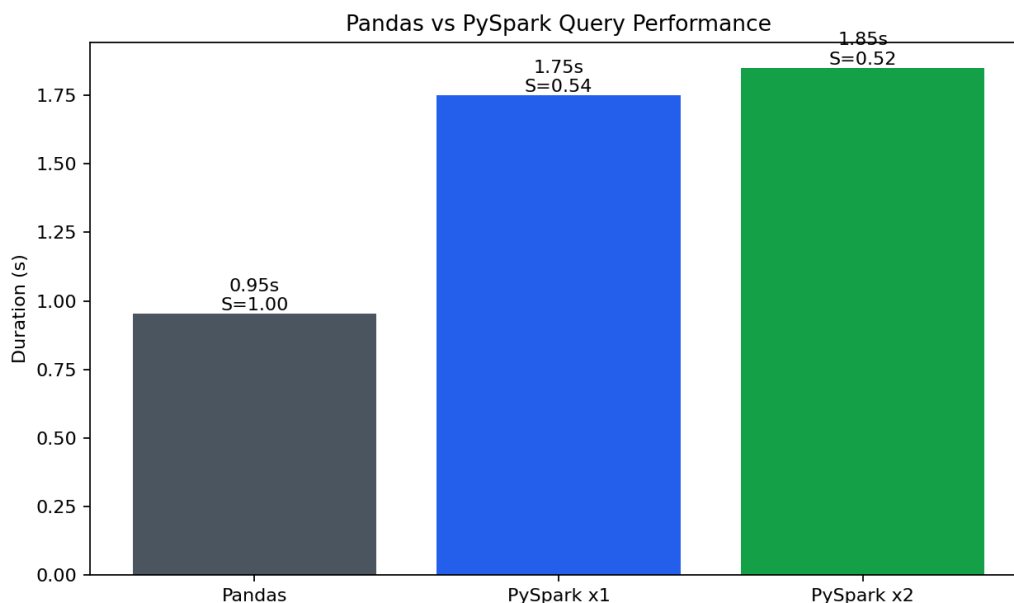


图9 Pandas 与 PySpark 查询性能对比图

该实验中，Pandas 反而比 Spark 更快。这并不矛盾，因为当前数据规模只有约 6 万条有效记录，远远不足以抵消 Spark 作业启动、任务调度、序列化和 Shuffle 所带来的固定开销。当 executor 从 1 增加到 2 后，总耗时没有下降，反而从 1.751s 略增到 1.851s，说明此时新增并行资源主要增加了协调成本，而没有换来足够的计算收益。

这与 Amdahl 定律完全一致。Amdahl 定律指出，当任务中串行部分和固定管理开销的比例较高时，即便继续增加并行度，也无法获得线性加速。对于本次实验而言，Q1 只是一次相对简单的聚合任务，输入规模较小，因此 Spark 的优势没有被体现出来。若数据规模扩展到数千万条甚至更高，或者分析流程包含更复杂的多阶段处理，Spark 的分布式收益才会更加明显。

附加题记录

附加题 1 监控系统

本组部署了基于 `kube-prometheus-stack` 的监控系统。组员补充截图显示，在 `monitoring` 命名空间下，Grafana、Prometheus、Alertmanager、node-exporter 与 operator 相关 Pod 均已进入 `Running` 状态，同时对应的 Service 也已创建完成，如图 图 1 所示。

```

PS C:\Users\11762\.kube> kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0  2/2     Running   0           96m
monitoring-grafana-6f45c677d7-86gyn  3/3     Running   0           96m
monitoring-kube-prometheus-operator-8586d5cfc9-knzbfc  1/1     Running   0           96m
monitoring-prometheus-node-exporter-679wg  1/1     Running   0           96m
monitoring-prometheus-node-exporter-85slj  1/1     Running   0           96m
monitoring-prometheus-node-exporter-95sxd  1/1     Running   0           96m
prometheus-monitoring-kube-prometheus-prometheus-0  2/2     Running   0           96m
PS C:\Users\11762\.kube> kubectl get svc -n monitoring
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)                                AGE
alertmanager-operated              ClusterIP    None          <none>         9093/TCP,9094/TCP,9094/UDP            96m
monitoring-grafana                 ClusterIP    10.247.135.76 <none>         80/TCP                                96m
monitoring-kube-prometheus-alertmanager  ClusterIP    10.247.166.200 <none>         9093/TCP,8080/TCP                    96m
monitoring-kube-prometheus-operator    ClusterIP    10.247.152.102 <none>         443/TCP                                96m
monitoring-kube-prometheus-prometheus  ClusterIP    10.247.96.8    <none>         9090/TCP,8080/TCP                    96m
monitoring-prometheus-node-exporter    ClusterIP    10.247.80.113  <none>         9100/TCP                               96m
prometheus-operated                 ClusterIP    None          <none>         9090/TCP                               96m

```

图1 monitoring 命名空间中监控组件运行状态

Grafana 仪表盘截图如图 2 所示。从中可以直接观察到三个典型指标：

- CPU Usage：展示不同命名空间随时间变化的 CPU 使用量；
- Memory：展示不同命名空间的内存占用情况；
- CPU Quota / Memory Requests：反映资源配额与资源请求配置。

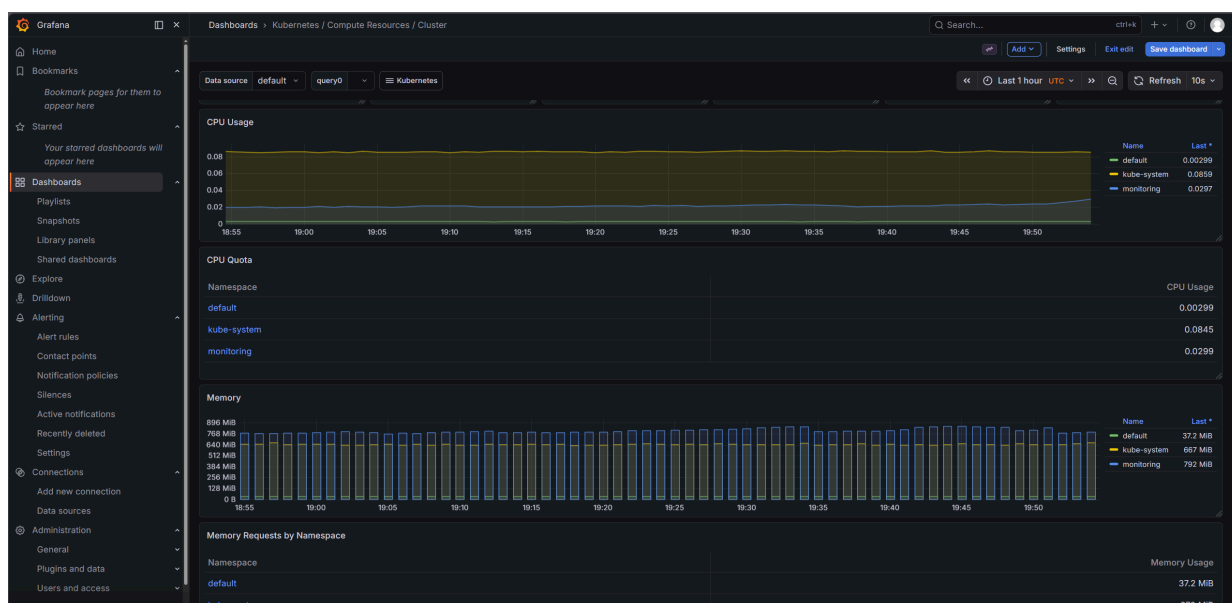


图2 Grafana 中的集群 CPU 与内存监控面板

Prometheus 采用 Pull 模式工作，即 Prometheus Server 按配置周期主动访问被监控对象暴露的 HTTP 指标端点，再将采集到的时间序列写入本地存储。这种方式便于统一服务发现、采集频率控制和目标可用性判断，因此非常适合 Kubernetes 集群这种动态资源环境。

附加题 2 CI/CD 流水线

仓库中已经新增 `.github/workflows/deploy.yml` 工作流文件。该流水线在 `main` 分支中 `scaffold/part1-app/**`、`scaffold/part1-k8s/**` 或工作流文件本身发生变更时自动触发，也支持手动触发。其主要步骤包括：

1. 检出代码；

2. 使用 `GITHUB_SHA` 生成镜像 tag;
3. 登录 SWR;
4. 构建 backend 和 frontend 镜像;
5. 推送镜像到 SWR;
6. 写入 kubeconfig;
7. 使用 `kubectl set image` 滚动更新 CCE 中的 Deployment。

组员补充的截图已经证明该流水线实际运行成功。图 图 3 展示了 GitHub Actions 中一次 Build and Deploy 执行记录：从 Checkout code、Login to SWR、Build and push backend image、Build and push frontend image 到 Update Kubernetes Deployment 均已成功完成。图 图 4 展示了滚动更新后的 Deployment 镜像地址已经变更为带提交哈希的新 tag，同时 `kubectl describe deployment backend` 中可以看到旧 ReplicaSet 被缩容、新 ReplicaSet 被拉起的事件记录。

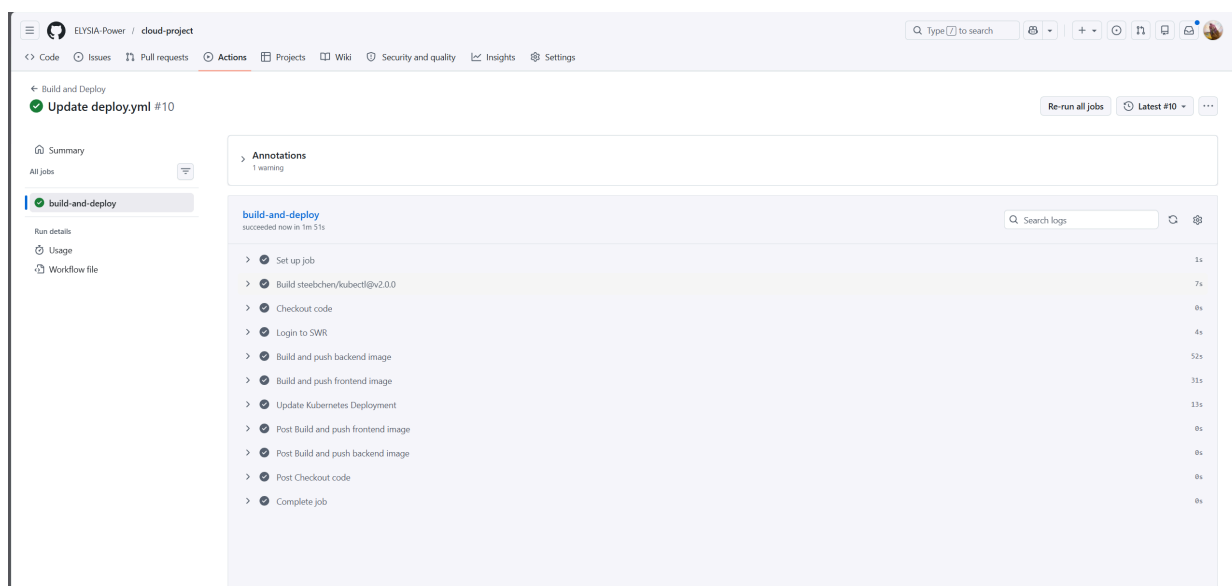


图 3 GitHub Actions 中 CI/CD 流水线执行成功

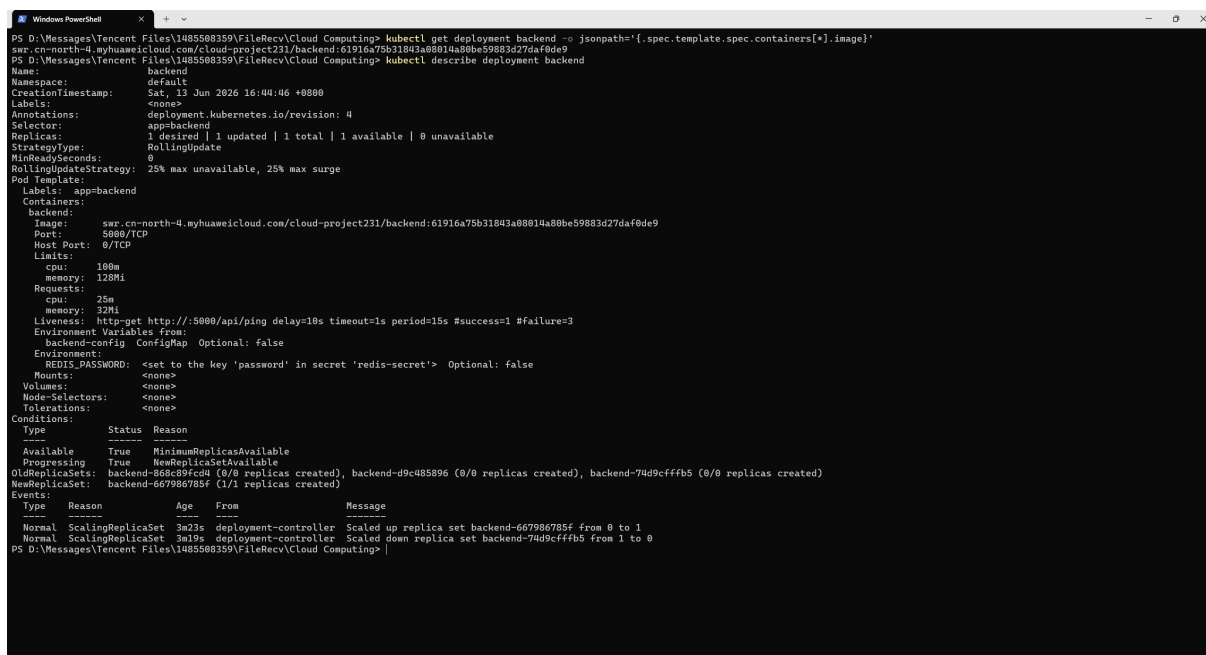


图 4 Kubernetes Deployment 镜像 tag 更新成功

该设计体现了最基本的 CI/CD 自动部署思路：开发者提交代码后，系统自动完成构建、制品生成和部署更新，从而降低人工重复操作的成本。其中 CI（持续集成）关注“代码是否能够被正确构建和验证”，对应本流水线中的检出、生成 tag、构建镜像等步骤；CD（持续部署）关注“通过验证的制品是否能够进入运行环境”，对应推送 SWR、写入 kubeconfig、更新 Deployment 和等待 rollout 成功等步骤。二者的边界在于：CI 产出可信制品，CD 将可信制品交付到目标环境。

GitOps 的核心理念是以 Git 仓库中的声明式配置作为系统期望状态，集群内控制器持续比较期望状态和实际状态，并自动完成同步。本实验中的 Actions 已经能把镜像 tag 更新到 CCE，但仍属于流水线直接操作集群的模式；如果进一步将新 tag 写回 YAML 或 Kustomize/Helm values，再由 Argo CD、Flux 等控制器负责同步，才是更完整的 GitOps 形态。这样做的优势是变更历史、回滚路径和审计记录都集中在 Git 中，适合多人协作和生产环境发布。

附加题 3 K3s + MQTT 边缘计算模拟

附加题 3 的实现目录为 `scaffold/addon-c2-mqtt/`，包括 `publisher.py`、`subscriber.py`、`cce-mosquitto.yaml`、`cce-subscriber.yaml` 与 `k3s-publisher.yaml`。实验链路为：边缘侧 K3s 发布模拟传感器消息，CCE 内的 Mosquitto Broker 接收消息，CCE 内 subscriber 持续订阅并写入 Redis。组员补充截图表明，左侧窗口为 subscriber 持续接收传感器消息，右侧窗口为 publisher 周期性发布消息，上方为 Broker 启动日志，如图 5 所示。

The figure shows a screenshot of a terminal window at the top displaying MQTT Broker logs: "MQTT Broker started on port 1883", "No data from (client @=127.0.0.1:4386) : No more data", and "Failed to initialize client session. No more data". Below this are two terminal windows. The left window shows the output of a subscriber, displaying a series of received JSON messages with fields like 'device_id', 'temperature', 'humidity', and 'timestamp'. The right window shows the output of a publisher, displaying a series of published JSON messages with the same fields. The messages are formatted as Python dictionaries.

图 5 K3s + MQTT 边缘计算模拟链路验证

从架构上看，本实验把云边协同拆成四个明确角色。第一，K3s 模拟边缘节点，运行 `mqtt-publisher`，每 2 秒生成一条温度、湿度和时间戳组成的 JSON 消息。第二，CCE 侧运行 Mosquitto Broker，并通过 `mosquitto-lb` 暴露 1883 端口，使边缘侧可以通过公网 ELB 建立 MQTT 连接。第三，CCE 侧运行 `mqtt-subscriber`，订阅 `edge/sensor` 主题，接收到消息后写入 Redis。第四，Redis 作为云端临时数据落地点，既保存按时间命名的键值，又通过 `mqtt_messages` 列表保留最近 50 条消息。这个链路对应了典型物联网场景中的“端侧采集、边侧轻量运行、云侧汇聚与存储”模式。

`publisher.py` 的核心逻辑较简单，但能体现 MQTT 在边缘侧的优势。程序从环境变量读取 `BROKER_HOST`、`BROKER_PORT`、`MQTT_TOPIC`、`INTERVAL_SECONDS` 和 `DEVICE_ID`，随后周期性构造 JSON payload，并以 `qos=1` 发布到 `edge/sensor`。QoS 1 的含义是“至少送达一次”，发送端会等待 broker 确认；在弱网环境下，这比纯 HTTP 一次性请求更适合处理短时丢包或链路抖动。另一方面，MQTT 使用长连接和发布/订阅模型，消息头部开销较小，

边缘设备不用知道云端有多少消费者，也不用为每个消费者维护独立连接。对于温湿度、门磁、电表、摄像头事件这类高频小包数据，MQTT 能降低带宽消耗和连接建立成本。

`subscriber.py` 体现了云端消费侧职责。程序连接 `mosquitto-svc`，订阅同一个 `edge/sensor` 主题，在 `on_message` 回调中将 `payload` 写入 Redis。这里既使用 `set(key, payload)` 保存按时间戳组织的消息，也使用 `lpush` 和 `ltrim` 维护最近 50 条消息列表，便于后续调试和页面展示。这个设计说明云端不必把 MQTT 消息直接作为最终状态，而可以把 broker 当作接入层，后面再接 Redis、时序数据库、流式计算或告警系统。实际生产中，如果消息用于实时告警，可以把 subscriber 替换为 Flink、Kafka Connect 或自定义规则引擎；如果消息用于趋势分析，则应写入时序数据库并配合 Grafana 展示。

CCE 侧的 `cce-mosquitto.yaml` 同时创建 ConfigMap、Deployment、ClusterIP Service 和 LoadBalancer Service。ConfigMap 中的 `mosquitto.conf` 设置 `listener 1883` 和 `allow_anonymous true`，这降低了课程实验部署门槛，但也暴露出安全限制：公网 1883 端口如果长期开放，任何知道地址的人都可以尝试连接 broker。`mosquitto-svc` 供集群内 subscriber 使用，`mosquitto-lb` 供 K3s 边缘 publisher 使用，这种双 Service 结构把内部消费和外部接入分开，符合 Kubernetes 中“内部服务名稳定、外部入口由云负载均衡承载”的设计方式。

边缘侧的 `k3s-publisher.yaml` 只部署一个 Pod，镜像与 subscriber 共用 `swr.cn-east-3.myhuaweicloud.com/cloud-ljh-ys/mqtt-edge:v1`，但启动命令改为 `python publisher.py`。发布端通过 `mqtt-edge-config` ConfigMap 读取云端 `mosquitto-lb` 的公网地址，避免把运行时公网 IP 固化在镜像中。这个细节也说明云边协同不是单纯“把程序放到两个集群里”就结束，还要处理跨网络寻址、服务发现和公网暴露问题。K3s 的价值在于它比完整 Kubernetes 更轻，适合安装在本地虚拟机、边缘网关或资源有限的小型服务器上，同时保留了 Pod、Service、镜像部署等 Kubernetes 编排能力。

从弱网适用性看，MQTT 的优势主要来自三个方面。第一，发布者和订阅者解耦：边缘设备只面向 broker 发布，不需要感知云端消费者是否扩缩容。第二，协议开销小：相比频繁 HTTP 请求，长连接和小消息头更适合移动网络、校园网 NAT 或边缘网关这种不稳定环境。第三，QoS 机制可按场景权衡可靠性和开销：本实验使用 QoS 1，能保证消息至少到达一次，但消费者侧必须考虑重复消息；若是普通环境监测，可接受少量重复并通过时间戳去重；若是强一致控制指令，还需要消息 ID、幂等写入和确认机制。实验截图中 publisher 与 subscriber 消息能够持续对应，说明在当前网络条件下链路可用。

云边协同的主要挑战在于延迟、可靠性和安全三方面。延迟方面，消息从 K3s Pod 到 CCE Broker 需要经过本地容器网络、宿主机网络、公网、华为云 ELB、CCE Service 和 Pod 网络，subscriber 写入 Redis 又增加了一次集群内访问，因此端到端延迟并不只由 MQTT 协议决定。可靠性方面，本实验中 broker 副本数为 1，且未配置持久化队列；如果 Mosquitto Pod 重启，正在传输或未持久化的消息可能丢失。安全方面，`allow_anonymous true` 和未启用 TLS 只适合课程验证，不适合真实公网部署。真实生产中应启用用户名密码、客户端证书、TLS 加密、ACL 主题权限，并限制 ELB 安全组来源。

如果继续完善该专题，我认为应从六个方向改进。第一，为 Mosquitto 增加持久化卷和持久会话，避免 broker 重启导致消息状态全部丢失。第二，为 publisher 增加本地缓存，当公网不可用时先写入本地文件或 SQLite，恢复连接后批量补发。第三，为每条消息加入 `message_id`，subscriber 写 Redis 时使用幂等键，解决 QoS 1 可能产生的重复消息。第四，将单 broker 改为高可用 MQTT 集群，或引入 EMQX 等更适合生产的 broker。第五，增加 Prometheus 指标，采集连接数、消息速率、订阅延迟和 Redis 写入失败次数。第六，在云端

增加规则引擎，例如温度超过阈值时触发告警，而不是只把消息写入 Redis。这样才能从“链路打通”进一步演进为可运维、可扩展、可审计的边缘计算系统。

本实验的局限也比较明确：截图证明了 publisher、broker、subscriber 三端可联通，但未单独保留 Redis `LRANGE mqtt_messages 0 5` 的截图；Mosquitto 配置为了降低实验复杂度而开放匿名访问；边缘端也没有模拟断网重连和消息补偿。因此，本专题更准确地说是“云边 MQTT 链路验证与方案分析”，不是完整生产级边缘平台。不过它已经覆盖了课程要求中的 K3s、MQTT、云端 K8s 接入和弱网适用性分析，也能说明边缘计算并非简单把服务下沉到本地，而是在网络不稳定、设备资源有限和云端集中处理之间做工程权衡。

问题与解决汇总

模块	问题或风险	处理方式
任务 1	前端宿主机 8080 端口被占用，Docker 无法绑定端口。	将 Compose 映射改为 18080:80，继续验证前端页面和 API。
任务 3	LoadBalancer 初始分配公网入口需要等待云端 ELB 创建。	通过 CCE 控制台和 <code>kubectl get svc</code> 双向确认 ELB 与 Service 绑定。
任务 4	Redis Pod 删除后容易误判为数据丢失。	使用 PVC 挂载 /data，按“SET、删 Pod、GET”三步截图验证。
任务 5	ConfigMap 文件挂载与环境变量注入容易混用。	Nginx 配置采用 Volume 挂载，Redis 地址继续用 <code>envFrom</code> 注入。
任务 6	HPA 扩缩容不是立即发生，停压后回收过程有稳定窗口。	保留扩容和回收过程截图，并在正文解释 metrics 周期与冷却窗口。
Spark	Driver/executor 初始资源请求过高导致 <code>FailedScheduling</code> 。	降低内存到 512m，设置 <code>coreRequest: 100m</code> ，并将数据打包入镜像。
附加题	外部镜像源和公网链路可能不稳定。	关键镜像推送至 SWR，CI/CD 与 MQTT 通过截图验证实际链路。

表 1 实验过程中的问题与解决方式汇总

总结与收获

本次课程设计将课堂中的 Docker、Kubernetes、Spark、Prometheus 和边缘消息队列等知识串联到了同一个实验闭环中。第一部分中，本组完成了本地三容器联调、SWR 镜像推送、CCE 集群部署、ELB 公网暴露、Redis PVC 持久化、ConfigMap 文件挂载和 HPA 弹性伸缩验证，形成了一条较完整的云原生应用交付路径。第二部分中，本组基于 Spark Operator 成功运行了 PySpark 数据分析任务，对豆瓣电影数据进行了缺失值清洗、聚合分析、Top-N 排序、时间趋势统计和窗口函数分组排名，并完成了 Pandas 与 Spark 的性能对比实验。

从实验结果来看，最有价值的收获并不只是“部署成功”，而是对云平台中各类组件职责边界的理解更加具体。例如，Service 与 ELB 负责访问入口，Deployment 负责副本管理，PVC 负责状态保存，ConfigMap 负责配置解耦，HPA 负责资源弹性，Spark Operator 负责将大数据任务纳入 Kubernetes 的统一调度体系。这些组件单独看都不复杂，但真正组合在一起后，才会暴露出诸如端口冲突、资源不足、凭证缺失、字段脏值和并行开销不划算等工程化问题。通过逐项排查与修正，本组对“课程知识如何落到真实系统”有了更直接的认识。

此外，性能对比实验也提醒我们，并行计算并不是越多副本越快，技术选型必须结合数据规模和任务结构来判断。当前数据规模下，Pandas 比 PySpark 更高效，说明分布式框架有其适用边界；而 HPA 的触发与回退过程也说明，所谓“弹性”本质上依赖于监控、控制器和资源调度共同配合。总体而言，本次课程设计既锻炼了云平台实操能力，也强化了对系统设计权衡、故障定位和实验报告证据化表达的理解。

附录 代码与仓库说明

本课程设计的完整代码仓库地址为：

```
1 https://github.com/Ljh998244353/cloud-compute-ex
```

text

正式提交时可将 PDF 与该仓库链接一并提交。仓库中与评分项对应的主要文件见表 1。

文件路径	对应任务	说明
scaffold/part1-app/backend/Dockerfile	任务 1	Flask 后端多阶段构建 Dockerfile。
scaffold/part1-app/backend/requirements.txt	任务 1	包含 flask 、 redis 和自选依赖 requests 。
scaffold/part1-app/frontend/Dockerfile	任务 1	Nginx 静态前端镜像构建文件。
scaffold/part1-app/frontend/static/index.html	任务 1	前端首页，包含学号与姓名。
scaffold/part1-app/docker-compose.yml	任务 1	本地 backend、frontend、redis 联调编排。
scaffold/part1-k8s/backend-deployment.yaml	任务 3/6	后端副本、资源限制、ConfigMap 与 Secret 注入。
scaffold/part1-k8s/redis-deployment.yaml	任务 3/4	Redis 部署、密码和 PVC 挂载。
scaffold/part1-k8s/service.yaml	任务 3	backend LoadBalancer 与 redis ClusterIP。
scaffold/part1-k8s/configmap-secret.yaml	任务 3	Redis 地址配置和 Redis Secret。
scaffold/part1-k8s/frontend-nginx-configmap.yaml	任务 5	Nginx 反向代理配置文件化挂载。
scaffold/part1-k8s/hpa.yaml	任务 6	backend HPA，1–4 副本，CPU 目标 60%。
scaffold/part2-spark/Dockerfile	A-0	PySpark 分析镜像构建文件。
scaffold/part2-spark/sparkapplication.yaml	A-0	SparkApplication 提交文件。
scaffold/part2-spark/analysis.py	A-1/A-2/A-3	数据清洗、4 个统计查询和性能计时。
scaffold/part2-spark/pandas_benchmark.py	A-3	Pandas 单机性能基准。

文件路径	对应任务	说明
<code>.github/workflows/ deploy.yml</code>	附加题 2	自动构建、推送 SWR、滚动更新 CCE Deployment。
<code>scaffold/addon-c2-mqtt/</code>	附加题 3	K3s + MQTT publisher、CCE subscriber 和 Mosquitto YAML。

表 1 提交仓库中主要代码与配置文件索引